Отчет по лабораторной работе №3
API Gateway с Nginx

Выполнил: студент группы 331 Иванов Иван
Вариант: tickets-s10

2024

Содержание

1 Цель работы **4**

2 Теоретическая часть **4**
2.1 Понятие API Gateway 4
2.2 Основные функции API Gateway 4
2.3 Почему Nginx? 4
2.4 Проблемы, решаемые API Gateway 5

3 Архитектура решения **5**
3.1 Общая архитектура 5
3.2 Спецификация варианта 5

4 Структура проекта **6**

5 Листинги кода **6**
5.1 Первый микросервис (tickets-service) 6
5.1.1 Файл `services/tickets-service/main.py` 6
5.1.2 Файл `services/tickets-service/requirements.txt` 7
5.1.3 Файл `services/tickets-service/Dockerfile` 7
5.2 Второй микросервис (other-service) 8
5.2.1 Файл `services/other-service/main.py` 8
5.2.2 Файл `services/other-service/requirements.txt` 9
5.2.3 Файл `services/other-service/Dockerfile` 9
5.3 Конфигурация Nginx (API Gateway) 9
5.3.1 Файл `nginx/nginx.conf` 9
5.4 Docker Compose 11
5.4.1 Файл `docker-compose.yml` 11

6 Инструкция по запуску **13**
6.1 Предварительные требования 13
6.2 Пошаговый запуск 13
6.3 Остановка проекта 13

7 Результаты тестирования **14**
7.1 Проверка доступности сервисов 14
7.2 Тестирование API через Gateway 14
7.2.1 Создание билета 14
7.2.2 Получение всех билетов 14
7.2.3 Создание элемента в other-service 15
7.2.4 Получение элементов other-service 15
7.3 Проверка обработки ошибок 15

8 Анализ работы API Gateway **15**
8.1 Преимущества использования Gateway 15
8.2 Сравнение прямого доступа и доступа через Gateway 16

9 Возможные проблемы и их решение **16**
9.1 Порт 80 уже занят 16
9.2 Контейнеры не запускаются 16
9.3 Ошибка разрешения имен 16

10 Выводы по лабораторной работе 17

1 Цель работы

Разработать API Gateway на базе Nginx для обеспечения единой точки входа к нескольким микросервисам. Настроить маршрутизацию запросов к соответствующим сервисам и скрыть внутреннюю структуру системы от клиента.

2 Теоретическая часть

2.1 Понятие API Gateway

API Gateway (шлюз API) - это сервер, который выступает единой точкой входа для всех клиентов микросервисной архитектуры. Он принимает запросы от клиентов и маршрутизирует их к соответствующим микросервисам. В современной микросервисной архитектуре API Gateway является обязательным компонентом, который решает множество проблем распределенных систем.

2.2 Основные функции API Gateway

- **Маршрутизация запросов** - направление запросов к нужным сервисам на основе пути, заголовков или других параметров
- **Единая точка входа** - клиент знает только один адрес, что упрощает разработку фронтенда
- **Балансировка нагрузки** - распределение запросов между несколькими экземплярами одного сервиса
- **Аутентификация и авторизация** - централизованная проверка прав доступа
- **Мониторинг и логирование** - сбор метрик со всех сервисов в одной точке
- **Трансформация запросов/ответов** - изменение формата данных при необходимости
- **Кэширование** - уменьшение нагрузки на бэкенд-сервисы

2.3 Почему Nginx?

Nginx выбран как API Gateway по следующим причинам:

- **Высокая производительность** - асинхронная архитектура позволяет обрабатывать тысячи одновременных соединений
- **Низкое потребление ресурсов** - минимальное использование памяти и CPU
- **Простая конфигурация** - декларативный язык конфигурации легко читается и поддерживается
- **Широкая распространенность** - используется в крупнейших проектах мира
- **Богатый функционал** - балансировка, кэширование, ограничение скорости и многое другое
- **Отличная документация** - большое сообщество и множество примеров

2.4 Проблемы, решаемые API Gateway

В распределенной системе без API Gateway клиент должен:

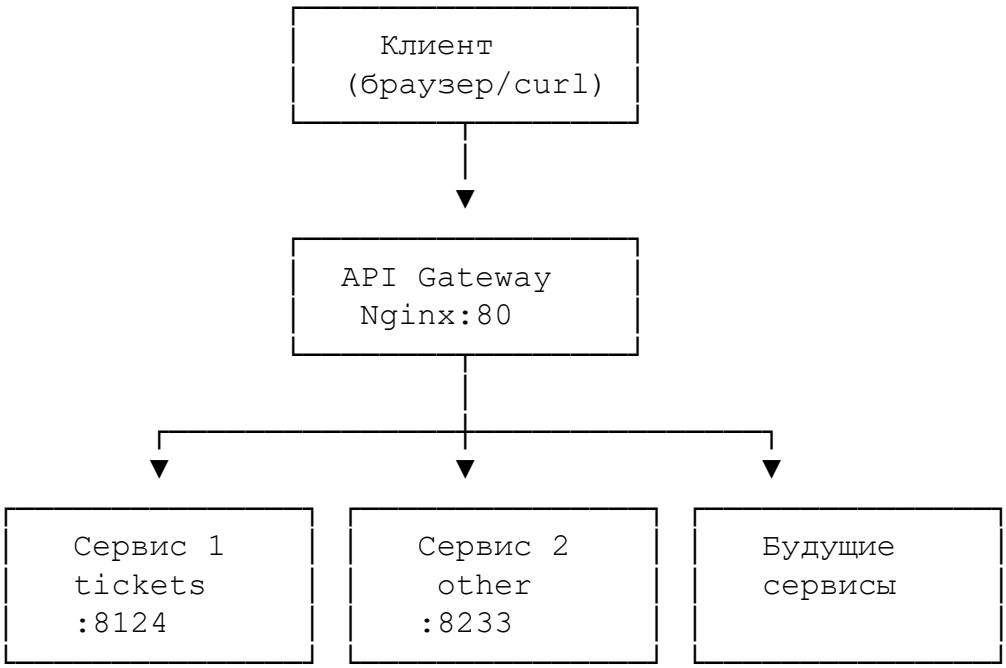
- Знать адреса всех микросервисов
- Управлять множеством соединений
- Реализовывать логику повторных попыток при ошибках
- Обрабатывать частичные отказы сервисов

API Gateway централизует эти задачи и предоставляет чистый интерфейс для клиентов.

3 Архитектура решения

3.1 Общая архитектура

В рамках лабораторной работы реализована следующая архитектура:



3.2 Спецификация варианта

Согласно варианту задания (tickets-s10):

- **Основной ресурс**: tickets (билеты)
- **Имя сервиса**: tickets-svc-s10
- **Порт сервиса**: 8124
- **Префикс Gateway**: /api
- **Маршрут**: /api/tickets → tickets-svc-s10

4 Структура проекта

Проект имеет следующую структуру файлов:

```
tickets-gateway-project/├──
 services/│
   ├── tickets-service/              # Основной сервис билетов│
   │   ├── main.py                    # FastAPI приложение│
   │   ├── requirements.txt            # Зависимости Python│
   │   └── Dockerfile                  # Инструкция для сборки образа│
   └── other-service/                  # Второй сервис для демонстрации│
       ├── main.py│
       ├── requirements.txt│
       └── Dockerfile├──
 nginx/│
   └── nginx.conf                      # Конфигурация API Gateway├──
 docker-compose.yml                   # Оркестрация всех контейнеров└──
 README.md                            # Документация по запуску
```

Листинг 1: Структура проекта API Gateway

5 Листинги кода

5.1 Первый микросервис (tickets-service)

5.1.1 Файл `services/tickets-service/main.py`

```python
from fastapi import FastAPI, HTTPException
from typing import List, Optional
from pydantic import BaseModel
from datetime import datetime

app = FastAPI(title="Tickets Microservice")

class Ticket(BaseModel):
    id: int
    event_name: str
    customer_name: str
    seat_number: str
    price: float
    created_at: datetime

class TicketCreate(BaseModel):
    event_name: str
    customer_name: str
    seat_number: str
    price: float

tickets_db = []
id_counter = 1

@app.post("/api/v1/tickets/", response_model=Ticket, status_code=201)
async def create_ticket(ticket: TicketCreate):
    """
    Создание нового билета.
    Принимает данные билета, присваивает ID и сохраняет в БД.
    """
    global id_counter
```

```python
    new_ticket = Ticket(
        id=id_counter,
        **ticket.model_dump(),
        created_at=datetime.now()
    )
    tickets_db.append(new_ticket)
    id_counter += 1
    return new_ticket

@app.get("/api/v1/tickets/", response_model=List[Ticket])
async def get_tickets():
    """
    Получение списка всех билетов.
    """
    return tickets_db

@app.get("/api/v1/tickets/{ticket_id}", response_model=Ticket)
async def get_ticket(ticket_id: int):
    """
    Получение конкретного билета по его ID.
    """
    ticket = next((t for t in tickets_db if t.id == ticket_id), None)
    if not ticket:
        raise HTTPException(status_code=404, detail="Билет не найден")
    return ticket

@app.delete("/api/v1/tickets/{ticket_id}")
async def delete_ticket(ticket_id: int):
    """
    Удаление билета по ID.
    """
    global tickets_db
    ticket = next((t for t in tickets_db if t.id == ticket_id), None)
    if not ticket:
        raise HTTPException(status_code=404, detail="Билет не найден")
    tickets_db = [t for t in tickets_db if t.id != ticket_id]
    return {"message": "Билет удален"}
```

Листинг 2: Основной сервис для управления билетами

5.1.2 Файл `services/tickets-service/requirements.txt`

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0
```

Листинг 3: Зависимости Python для tickets-service

5.1.3 Файл `services/tickets-service/Dockerfile`

```dockerfile
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY *.py .
```

```

EXPOSE 8124

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8124"]
```

Листинг 4: Dockerfile для tickets-service

5.2 Второй микросервис (other-service)

5.2.1 Файл `services/other-service/main.py`

```
from fastapi import FastAPI, HTTPException
from typing import List
from pydantic import BaseModel
from datetime import datetime

app = FastAPI(title="Other Service")

class OtherItem(BaseModel):
    id: int
    name: str
    description: str
    created_at: datetime

other_db = [
    OtherItem(
        id=1,
        name="Документация API",
        description="Описание всех endpoints",
        created_at=datetime.now()
    ),
    OtherItem(
        id=2,
        name="Логи сервера",
        description="Логи за последний час",
        created_at=datetime.now()
    ),
]
id_counter = 3

@app.get("/api/v1/other/", response_model=List[OtherItem])
async def get_other_items():
    """
    Получение списка всех элементов.
    """
    return other_db

@app.get("/api/v1/other/{item_id}", response_model=OtherItem)
async def get_other_item(item_id: int):
    """
    Получение конкретного элемента по ID.
    """
    item = next((i for i in other_db if i.id == item_id), None)
    if not item:
        raise HTTPException(status_code=404, detail="Элемент не найден")
    return item

@app.post("/api/v1/other/", response_model=OtherItem, status_code=201)
async def create_other_item(name: str, description: str):
```

```python
    """
    Создание нового элемента.
    """
    global id_counter
    new_item = OtherItem(
        id=id_counter,
        name=name,
        description=description,
        created_at=datetime.now()
    )
    other_db.append(new_item)
    id_counter += 1
    return new_item

@app.delete("/api/v1/other/{item_id}")
async def delete_other_item(item_id: int):
    """
    Удаление элемента по ID.
    """
    global other_db
    item = next((i for i in other_db if i.id == item_id), None)
    if not item:
        raise HTTPException(status_code=404, detail="Элемент не найден")
    other_db = [i for i in other_db if i.id != item_id]
    return {"message": "Элемент удален"}
```

Листинг 5: Дополнительный сервис для демонстрации

5.2.2 Файл `services/other-service/requirements.txt`

```
fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0
```

Листинг 6: Зависимости Python для other-service

5.2.3 Файл `services/other-service/Dockerfile`

```dockerfile
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY *.py .

EXPOSE 8233

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8233"]
```

Листинг 7: Dockerfile для other-service

5.3 Конфигурация Nginx (API Gateway)

5.3.1 Файл `nginx/nginx.conf`

```
events {
    worker_connections 1024;
}

http {
    # Формат логирования
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log;

    # Определение групп серверов (upstream)
    upstream tickets_service {
        server tickets-service:8124;
    }

    upstream other_service {
        server other-service:8233;
    }

    server {
        listen 80;
        server_name localhost;

        # ===== МАРШРУТЫ ДЛЯ СЕРВИСА БИЛЕТОВ =====

        # Маршрут для /api/v1/tickets без( слеша)
        location /api/v1/tickets {
            proxy_pass http://tickets_service/api/v1/tickets;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # Маршрут для /api/v1/tickets/ со( слешем)
        location /api/v1/tickets/ {
            proxy_pass http://tickets_service/api/v1/tickets/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # ===== МАРШРУТЫ ДЛЯ OTHER СЕРВИСА =====

        location /api/v1/other {
            proxy_pass http://other_service/api/v1/other;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location /api/v1/other/ {
            proxy_pass http://other_service/api/v1/other/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
```

```
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # ===== МАРШРУТЫ ДЛЯ SWAGGER ДОКУМЕНТАЦИИ =====

        location /api/v1/tickets/docs {
            proxy_pass http://tickets_service/api/v1/tickets/docs;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /api/v1/other/docs {
            proxy_pass http://other_service/api/v1/other/docs;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        # ===== ОБРАБОТКА НЕСУЩЕСТВУЮЩИХ ПУТЕЙ =====

        location / {
            return 404;
        }
    }
}
```

Листинг 8: Конфигурация Nginx как API Gateway

Пояснение ключевых директив Nginx:

- `upstream` - определяет группу серверов (в нашем случае по одному сервису)
- `proxy_pass` - перенаправляет запрос на указанный upstream сервер
- `proxy_set_header` - добавляет/изменяет заголовки при проксировании
- `Host $host` - передает оригинальный Host заголовок
- `X-Real-IP` - передает реальный IP клиента
- `location` - определяет правила сопоставления путей

5.4 Docker Compose

5.4.1 Файл `docker-compose.yml`

```
version: '3.8'

services:
  # Первый микросервис - управление билетами
  tickets-service:
    build: ./services/tickets-service
    container_name: tickets-svc-s10
    ports:
      - "8124:8124"  # Проброс порта для прямого доступа опционально()
    networks:
      - gateway-network
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8124/api/v1/tickets/"]
```

```
      interval: 30s
      timeout: 10s
      retries: 3

  # Второй микросервис - дополнительный сервис
  other-service:
    build: ./services/other-service
    container_name: other-svc
    ports:
      - "8233:8233"  # Проброс порта для прямого доступа опционально()
    networks:
      - gateway-network
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8233/api/v1/other/"]
      interval: 30s
      timeout: 10s
      retries: 3

  # API Gateway на базе Nginx
  nginx-gateway:
    image: nginx:latest
    container_name: api-gateway
    ports:
      - "80:80"  # Единая точка входа для клиентов
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      tickets-service:
        condition: service_healthy
      other-service:
        condition: service_healthy
    networks:
      - gateway-network
    restart: unless-stopped

# Общая сеть для взаимодействия сервисов
networks:
  gateway-network:
    driver: bridge
```

Листинг 9: Оркестрация всех сервисов с Docker Compose

Пояснение Docker Compose:

- `build` - указывает путь к Dockerfile для сборки образа
- `ports` - пробрасывает порты из контейнера на хост-машину
- `networks` - все сервисы в одной сети для взаимодействия
- `depends_on` - определяет порядок запуска сервисов
- `healthcheck` - проверяет готовность сервиса к работе
- `volumes` - монтирует конфигурацию Nginx из файла

6 Инструкция по запуску

6.1 Предварительные требования

- Установленный Docker (`docker --version`)
- Установленный Docker Compose (`docker-compose --version`)
- 1 ГБ свободной оперативной памяти
- 5 ГБ свободного места на диске

6.2 Пошаговый запуск

1. **Клонирование/создание проекта**

```
mkdir tickets-gateway-project
cd tickets-gateway-project
# Создайте все файлы согласно структуре выше

```

2. **Запуск всех сервисов**

```
# Запуск с отображением логов
docker-compose up --build

# Или запуск в фоновом режиме
docker-compose up -d --build

```

3. **Проверка запущенных контейнеров**

```
docker ps

```

   Должны увидеть 3 контейнера:

   - `tickets-svc-s10`
   - `other-svc`
   - `api-gateway`

4. **Просмотр логов**

```
# Логи всех сервисов
docker-compose logs -f

# Логи конкретного сервиса
docker-compose logs tickets-service
docker-compose logs nginx-gateway

```

6.3 Остановка проекта

```
# Остановка с сохранением данных
docker-compose down

# Остановка с полной очисткой удаление( томов)
docker-compose down -v
```

7 Результаты тестирования

7.1 Проверка доступности сервисов

Сервис	Прямой доступ	Через Gateway	Статус
Tickets Service	http://localhost:8124/api/v1/tickets/	http://localhost/api/v1/tickets/	□ Дост
Other Service	http://localhost:8233/api/v1/other/	http://localhost/api/v1/other/	□ Дост
Документация Tickets	http://localhost:8124/docs	http://localhost/api/v1/tickets/docs	□ Дост
Документация Other	http://localhost:8233/docs	http://localhost/api/v1/other/docs	□ Дост

Таблица 1: Проверка доступности сервисов

7.2 Тестирование API через Gateway

7.2.1 Создание билета

```
curl -X POST http://localhost/api/v1/tickets/ \
  -H "Content-Type: application/json" \
  -d '{
    "event_name": Концерт" группы",
    "customer_name": Иван" Петров",
    "seat_number": "A1",
    "price": 2500
  }'
```

Листинг 10: Запрос на создание билета

```
{
  "id": 1,
  "event_name": Концерт" группы",
  "customer_name": Иван" Петров",
  "seat_number": "A1",
  "price": 2500.0,
  "created_at": "2024-03-15T10:30:00.123456"
}
```

Листинг 11: Ответ сервера

7.2.2 Получение всех билетов

```
curl http://localhost/api/v1/tickets/
```

Листинг 12: Запрос на получение всех билетов

```
[
  {
    "id": 1,
    "event_name": Концерт" группы",
    "customer_name": Иван" Петров",
    "seat_number": "A1",
    "price": 2500.0,
    "created_at": "2024-03-15T10:30:00.123456"
  }
]
```

Листинг 13: Ответ сервера

7.2.3 Создание элемента в other-service

```
curl -X POST "http://localhost/api/v1/other/?nameТестовыйэлемент=%20&description
    Созданчерез=%20%20Gateway"
```

Листинг 14: Запрос на создание элемента

```
{
  "id": 3,
  "name": Тестовый" элемент",
  "description": Создан" через Gateway",
  "created_at": "2024-03-15T10:35:00.123456"
}
```

Листинг 15: Ответ сервера

7.2.4 Получение элементов other-service

```
curl http://localhost/api/v1/other/
```

Листинг 16: Запрос на получение всех элементов

7.3 Проверка обработки ошибок

```
curl -I http://localhost/api/v1/nonexistent
```

Листинг 17: Запрос к несуществующему пути

```
HTTP/1.1 404 Not Found
Server: nginx/1.24.0
Content-Type: application/json
Content-Length: 0
```

Листинг 18: Ответ сервера (заголовки)

8 Анализ работы API Gateway

8.1 Преимущества использования Gateway

1. **Упрощение клиентской части**
   - Клиенту нужно знать только один адрес (localhost:80)
   - Не требуется управлять множеством соединений
   - Изменения в архитектуре бэкенда не влияют на клиента
2. **Централизованное управление**
   - Единая точка для применения политик безопасности
   - Централизованное логирование всех запросов
   - Возможность добавить аутентификацию для всех сервисов сразу
3. **Масштабируемость**

- Легко добавить новые сервисы (достаточно обновить nginx.conf)
- Возможность балансировки нагрузки между экземплярами
- Независимое масштабирование сервисов

8.2 Сравнение прямого доступа и доступа через Gateway

Критерий	Прямой доступ	Через Gateway
Количество URL для клиента	Много (по числу сервисов)	Один
Сложность клиента	Высокая	Низкая
Централизованный мониторинг	Нет	Да
Возможность балансировки	Требует реализации	Встроена
Изменение архитектуры	Требует изменений клиента	Прозрачно для клиента

Таблица 2: Сравнение подходов к доступу к микросервисам

9 Возможные проблемы и их решение

9.1 Порт 80 уже занят

```
ports:
  - "8080:80"  # Использовать порт 8080 вместо 80
```

Листинг 19: Решение - изменить порт в docker-compose.yml

9.2 Контейнеры не запускаются

```
# Проверка логов
docker-compose logs

# Полная пересборка
docker-compose down -v
docker-compose up --build
```

Листинг 20: Диагностика и решение

9.3 Ошибка разрешения имен

```
# Проверка сети
docker network ls
docker network inspect tickets-gateway-project_gateway-network

# Проверка связи между контейнерами
docker exec api-gateway ping tickets-service
```

Листинг 21: Проверка сети

10 Выводы по лабораторной работе

В ходе выполнения лабораторной работы №3 был успешно реализован API Gateway на базе Nginx, который обеспечивает:

1. **Единую точку входа** для всех микросервисов на порту 80. Клиент (фронтенд, мобильное приложение) взаимодействует только с Gateway, что значительно упрощает разработку клиентской части.

2. **Правильную маршрутизацию** запросов к соответствующим сервисам:

   - Запросы на /api/v1/tickets → tickets-service (порт 8124)
   - Запросы на /api/v1/other → other-service (порт 8233)

3. **Прозрачность архитектуры** - клиент не знает о внутренней структуре системы. Это позволяет изменять количество и расположение микросервисов без необходимости обновления клиентского кода.

4. **Масштабируемость** - архитектура легко расширяется добавлением новых сервисов. Для добавления нового сервиса достаточно обновить конфигурацию Nginx и перезапустить Gateway.

5. **Централизованное управление** - все запросы проходят через единую точку, что упрощает мониторинг, логирование и обеспечение безопасности.

Практическая значимость работы заключается в понимании того, как API Gateway решает ключевые проблемы микросервисной архитектуры: сложность для клиента, необходимость знать адреса всех сервисов, отсутствие единой точки для мониторинга и управления.

Полученные навыки могут быть применены в реальных проектах для построения отказоустойчивых и масштабируемых микросервисных систем.